

DevIntent: How Much Does LLM-generated Code Violate Developer Intent?

Anonymous ACL submission

Abstract

Code generated by LLMs can violate a developer’s implicit intentions when given an ambiguous prompt, yet standard benchmarks measure only whether code passes its stated test. We introduce the Intent Violation Rate (IVR) and a 49-problem pilot benchmark derived from HumanEval+. Each problem strips implicit constraints from a clarified prompt and encodes them as hidden constraint tests. IVR measures the fraction of LLM-generated solutions that pass the stated (visible) tests yet fail hidden constraint tests that capture unstated intent. Evaluating Claude Sonnet 4.6 and OpenAI GPT 4.1, we find both pass over 92% of stated tests yet violate intent in over half of problems (54.5% and 63.5%), following a systematic, bi-modal pattern consistent across both models. Our findings indicate that pass rates overstate how well generated code reflects developer intent.

1 Introduction

When LLMs are tasked with building code, they can generate code sufficient for the main task at hand, such as writing a function or script. However, these outputs are not always fully correct, requiring human-in-the-loop review and adjustments before they can be deployed. Developers, through review, will capture functional or logical errors, and cases where the code does something different from what they intended. Current benchmarks measure functional correctness of LLM-generated code via a test pass rate (Austin et al., 2021), however, this metric does not capture whether the solution fully reflects what the developer intended.

Test pass rates alone are insufficient as a metric because they do not detect intent violations, which refer to scenarios where LLMs will provide outputs that satisfy stated tests, but violate implicit constraints the developer assumes will be accounted for, and did not explicitly state. Prior work has documented similar failures in which LLMs produce

code that will game specifications and exploit evaluation loopholes to pass tests without solving the intended task (Zhong et al., 2025). We focus on a distinct failure mode: outputs that legitimately pass valid stated tests, but still violate unstated developer intent. To measure this, we propose the Intent Violation Rate (IVR), a metric that quantifies whether an output violates developer intent. We evaluate IVR on 49 algorithm specification problems, finding that across two models, they violate intent in more than half of the cases, with Sonnet 4.6 violating implicit developer constraints in approximately 54.5% of cases, and GPT 4.1 in 63.5%, even when solutions pass the stated tests.

2 Related Work

A key concern is when developers take the outputted code at face value due to a high pass rate on tests they built. Prior work defines code that passes provided test cases as having functional correctness (Austin et al., 2021), however Chen et al. (2021) also note that models can pass weak test suites simply by deleting failing functionality. Models may also "overfit to assert statements", rather than properly apply logic (Austin et al., 2021), hardcoding outputs for visible tests rather than create an accurate function that properly handled the general task at hand. A growing body of work also details LLMs that exploit evaluation shortcuts in code generation; this is seen in test suites where tasks were constructed to purposefully contradict specifications (Zhong et al., 2025), or where outputs are hardcoded, or tests were edited to pass (Gabor et al., 2025). Despite the focus on deliberate exploitation of flawed or gameable test suites, they further show that solutions passing held-out tests can still be incorrect. Recent benchmarks also expand evaluation beyond basic generation, such as LiveCodeBench (Jain et al., 2024), which adds tasks like self-repair and code execution while ad-

082 dressing contamination through time-segmented
083 problem collection. These efforts nonetheless mea-
084 sure functional correctness, and we build on this
085 observation to measure violations of unstated intent
086 in generated code that passes valid stated tests.

087 By default, models rarely ask for clarification
088 when a prompt is unclear, so when it comes to
089 giving LLMs ambiguous prompts, prior work has
090 seen a drop in test pass rates by approximately
091 80% (Li et al., 2026). It shows that functional cor-
092 rectness (Pass@1) will fall apart when ambiguity
093 is introduced to prompts, and we measure the in-
094 tent violated in code where functional correctness
095 holds. However, to address the drop in test pass
096 rates, other work finds that asking clarifying ques-
097 tions improves test pass rates (Mu et al., 2023),
098 and ambiguity-aware training improves detection
099 (Kim et al., 2024). Rather than utilize methods to
100 resolve ambiguity through interaction or prompt
101 repair, IVR measures intent violations that remain
102 in code that passes its stated tests, generated from
103 ambiguous prompts.

104 3 Measuring Intent Violation Rate

105 3.1 Definition

106 Intent Violation Rate, or IVR, is defined as follows:

$$107 \text{IVR} = \frac{n_{\text{violating}}}{n_{\text{stated passed}}} \quad (1)$$

108 Where for a given solution:

109 $n_{\text{stated passed}}$: number of generated solutions pass-
110 ing all stated tests

111 $n_{\text{violating}}$: subset of those that fail ≥ 1 hidden con-
112 straint test

113 IVR can be interpreted as: "Of the solutions that
114 satisfy the given spec's visible tests, what fraction
115 violates the developer's actual intent?" It is only
116 measured over solutions that pass visible stated
117 tests, therefore any solutions that do not pass the
118 initial public test are not included in the calculation.
119 The reported IVR is the mean of per-problem IVRs
120 over problems that had ≥ 1 solution passing stated
121 test C1.

122 3.2 Benchmark Construction

123 The built benchmark consists of 49 algorithm prob-
124 lems derived from HumanEval+ (Liu et al., 2023).
125 Each problem consists of an ambiguous prompt,

126 a gold prompt, a stated test (C1), and hidden con-
127 straint tests (C2–C4, the number varying per prob-
128 lem).

129 The selected HumanEval+ problems all con-
130 tained a gold prompt, which we stripped down
131 to produce an ambiguous prompt. In some cases,
132 additional specifics were added to the gold prompt
133 to further clarify the task at hand. This gold prompt
134 was always hidden to the LLM, as it only received
135 the ambiguous prompt, and C1, the stated test.
136 Each aspect that was stripped from the gold prompt
137 to formulate the ambiguous prompt was then used
138 to create the hidden constraint tests.

139 Ambiguous prompts were constructed manually
140 by the authors, removing explicitly stated con-
141 straints from each gold prompt while preserving
142 the core task description. For a subset of problems,
143 a Claude LLM was used as a drafting aid for the
144 C2-C4 hidden constraint tests in cases where it
145 was unclear whether a hand-written test adequately
146 captured the intended constraint. In all cases, the
147 authors manually reviewed and validated the final
148 tests, and all tests pass the validation checks de-
149 scribed in Section 3.2

150 The scope was narrowed down to algorithm prob-
151 lems, where the prompt described a self contained
152 function. A subset of these problems were cleanly
153 decomposable into executable tests, and the for-
154 mulation of the ambiguous prompt came from un-
155 der specifying rather than misleading. Other task
156 types, such as data transformation, involve fuzzier
157 constraints and lack a comparable source at scale,
158 being left to future work.

159 As an example, the original HumanEval/34
160 prompt reads: "Return sorted unique elements in a
161 list."

We decompose it as shown in Table 1.

HumanEval/34	
Ambig.	Return the unique elements in a list.
Gold	Return only the unique values, sorted in ascending order.
C1	<i>stated</i> : only unique values solution([-2, 4, 4, 6, 6]) == [-2, 4, 6]
C2	<i>hidden</i> : sorted ascending solution([-2, 0, 5, 3, 5, 0, 3]) == [-2, 0, 3, 5]

Table 1: Example spec pair. Stripping the sort-order requirement from the gold prompt yields the ambiguous prompt; C1 tests what remains stated, C2 tests the stripped intent.

To ensure the benchmark is sound, we applied two validation checks. First, a leakage audit was performed. When building the stated test, it must be satisfiable from the ambiguous prompt alone. During initial construction, 11 of the 49 problems failed this check. 9 of the failed tests were due to type errors, output format contamination, and hidden constraints leaking into the stated test. Of the hidden constraint leakage, what occurred was that the stated test also check a hidden constraint test, so a solution could not pass without already knowing part of the developer’s intent. The other 2 problems were identified in the second validation check, where the canonical solution was run against the entire test suite to ensure stated and hidden tests were sound. These issues were errors in the tests themselves, something the initial leakage audit could not catch. Across all 49 problems, the validated references yield an IVR = 0, confirming that the hidden tests are satisfiable by correct code, and are not themselves the source of the measured violations.

3.3 Evaluation Pipeline

The pipeline occurs in 4 stages: generation, execution, pass/fail determination, and IVR calculation.

Generation: Solutions are generated from the ambiguous prompt and the stated test C1, using Claude Sonnet 4.6 and GPT 4.1, both at the API default temperature of 1.0. All other information (e.g., gold clarified prompt and hidden tests) is not shown to the LLM.

Execution: For each problem, 5 solutions are produced, with a 5-second timeout per test, returning a pass/fail result. This is adapted from HumanEval’s own execution pipeline, which runs candidate solutions in process isolation with syscall lockdown and timeouts (Chen et al., 2021).

Pass/Fail Determination: Each solution for each problem is run against the stated test C1 and hidden tests (C2–C4), producing a per-solution result for each problem. IVR is then calculated for that specific problem.

Overall IVR Calculation: Each problem produces its own IVR value, and the overall IVR value is the mean across N problems with at least one stated-test-passing solution.

95% confidence intervals are also reported, estimated via bootstrap resampling over problems (10,000 iterations, seeded).

4 Experiments

4.1 Setup

We evaluated Sonnet 4.6 and GPT-4.1 following the pipeline in Section 3.3, generating 5 solutions per problem at temperature 1.0 with a 1024-token limit. This is a dual-model evaluation; we leave additional models for future work.

4.2 Results

Across the stated tests, Sonnet 4.6 passes 94.3% of the time, but violates at least one hidden intent constraint in 54.5% of problems (95% CI: 40.4–68.1%); GPT-4.1 passes 92.7% but violates in 63.5% (95% CI: 50.0–76.5%). Both models exhibit the high functional correctness of LLM-generated code that nonetheless violates unstated intent. Of the 49 benchmark problems, some between the two models (Sonnet 4.6: HE/101, HE/112; GPT-4.1: HE/21, HE/108, HE/112) had no solution passing the stated test C1 and are therefore excluded from the IVR calculation per model, leaving N=47 for Sonnet 4.6 and N=46 for GPT-4.1. The confidence intervals overlap, so the two model’s violation rates are not statistically distinguishable at this sample size, though substantial.

Metric	Claude Sonnet 4.6	GPT-4.1
C1 pass rate	94.3%	92.7%
IVR	54.5%	63.5%
95% CI	40.4–68.1%	50.0–76.5%
Qualifying Problems	47	46
% at IVR poles	95.7%	91.3%

Table 2: IVR results for both models across the 49 spec pairs (5 solutions per problem). Qualifying problems have ≥ 1 stated-test-passing solution (excluded: HE/101, HE/112 for Claude; HE/21, HE/108, HE/112 for GPT-4.1). Per-problem IVR concentrates at the extremes (0 or 1) for both models.

Problems tended to cluster at IVR = 0 (no solutions violate intent) or IVR = 1 (every solution violates intent), with 45 of 47 (95.7%) for Sonnet 4.6 (Figure 1) and 42 of 46 (91.3%) for GPT-4.1 (Figure 2) at these extremes, showing a bimodal distribution for both models. This near-deterministic behavior indicates intent violations are systematic, not sampling noise. If violations were due to random variation in generation, per-problem IVR would spread across intermediate values. Instead, both independently trained models produce outputs

that either fail to or fully capture hidden constraints.

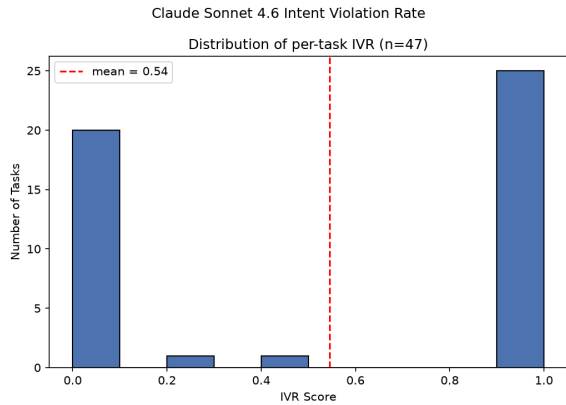


Figure 1: Distribution of per-problem IVR for Claude Sonnet 4.6 across 47 spec pairs. Two problems with no stated-test-passing solutions are excluded.

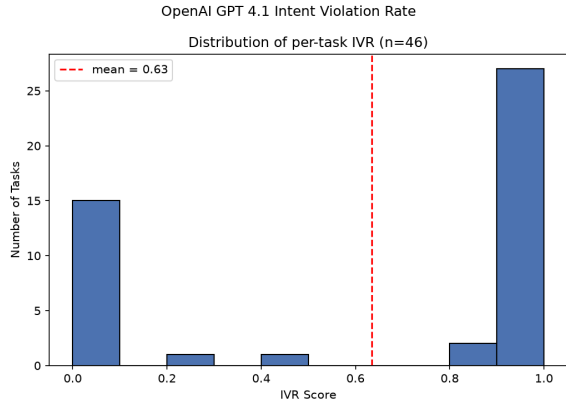


Figure 2: Distribution of per-problem IVR for GPT 4.1 across 46 spec pairs. Three problems with no stated-test-passing solutions are excluded.

Both models rarely fail stated test C1 much less often than the hidden constraints, at 5.7% for Sonnet 4.6 and 7.3% for GPT-4.1. Among the hidden constraints, fail rates decline monotonically from C2 to C4 for each model (Table 3): from 42.9% to 15.4% for Sonnet 4.6 and from 44.1% to 21.5% for GPT-4.1. GPT-4.1’s fail rates are somewhat higher, however, the overall difference is not significant.

4.3 Analysis

Hidden constraints are numbered in the order they were decomposed from the gold prompt, and this ordering is positional and does not strictly rank constraints by centrality. We nonetheless observe that earlier numbered constraints fail more often than later ones for both models (Table 3), which may indicate that the first stripped constraint is often most central to the task.

Constraint	Present	Failed	Fail Rate
<i>Claude Sonnet 4.6</i>			
C1 (stated)	245	14	5.7%
C2 (hidden)	231	99	42.9%
C3 (hidden)	201	58	28.9%
C4 (hidden)	65	10	15.4%
<i>GPT-4.1</i>			
C1 (stated)	245	18	7.3%
C2 (hidden)	227	100	44.1%
C3 (hidden)	199	69	34.7%
C4 (hidden)	65	14	21.5%

Table 3: Failure rate by constraint for both models across the 49 spec pairs (245 solutions each). “Present” counts solutions whose problem included the constraint; C2–C4 rates are over stated-test-passing solutions.

In HumanEval/34 (Section 3.2), the IVR of 1.0 for both models, with every stated-test-passing solution failing hidden ascending-sort order constraint C2. However, in HumanEval/25, the ambiguous prompt reads as "Write a function named solution that returns the prime factors of a given integer", and it contains the same hidden constraint of returning the output in ascending order. Across both models, all five solutions pass the stated and hidden constraints, yielding an IVR of 0.0, as standard factorization naturally produces factors in ascending order. This suggests intent satisfaction is constraint and context-dependent.

5 Conclusion

We introduced the Intent Violation Rate (IVR), a metric that measures how often LLM-generated code passes its stated tests while violating unstated developer intent, and a small pilot benchmark of N=49 audited algorithm specification pairs derived from HumanEval+. Evaluating Claude Sonnet 4.6 and GPT 4.1 on this benchmark, despite high stated-test pass rates (94.3% and 92.7%), both violated intent in over half of the problems (54.5% and 63.5%). These violations fell into a systematic and near-deterministic pattern in both models rather than random noise, indicating that functional correctness metrics can overstate how well generated code reflects what a developer actually intended. Future work includes underspecification detection, expanding the benchmark, and extending IVR across additional models and task domains.

Limitations

Our experiment and benchmarking were only tested on the Sonnet 4.6 and GPT-4.1 models, at a single temperature of 1.0 and max tokens of 1024. IVR could differ across other models, such as Llama or Qwen and different settings such as lower temperatures. Additionally, the benchmark is small, at only $N=49$, limiting statistical power, with constraint-type distributions being more suggestive than robust. The overlapping confidence intervals also mean we cannot distinguish the two models' violation rates.

Per-problem IVR is computed over only 5 samples, so it takes on one of six discrete values. This coarse resolution limits how precisely we can characterize the distribution's shape, however the strong concentration as the extremes suggest clustering is not solely an artifact in Section 4.2.

A Claude model was used to assist in drafting a subset of the constraint tests, though all tests were manually reviewed and validated by the authors and passed our validation checks (Section 3.2). Due to this, Claude Sonnet 4.6's measured IVR could be biased downward, toward passing tests it may have helped shape, making its reported rate a conservative estimate. GPT-4.1, which played no role in test construction, serves as an independent control, exhibiting a comparable and higher IVR on the same tests, indicating that the measured violations are not an artifact of test authorship.

The benchmark only focuses on Python algorithmic problems as other languages and problem types like data transformation may yield different findings. The problem specification is constrained to single-annotator construction, therefore the defined behavior of clarified gold prompts and tests reflects the judgment of one person. This also affects the operationalized definition of "intent", as we do not measure "intent" in an absolute metric. Also, HumanEval+ is a public dataset, so it is possible that the models have seen the original problems in training prior to our experimentation.

Ethical Considerations

Our work highlights a failure mode with direct deployment implications: LLM-generated code can pass its stated tests while silently violating unstated developer intent. As LLM-assisted coding is increasingly integrated into software workflows, a high test pass rate may give developers false confidence in code that does not do what they

intended. We frame IVR as a measurement that makes this gap measurable, and we caution against treating functional-correctness metrics as sufficient evidence of correctness in deployment.

Our benchmark is derived from HumanEval+ (Liu et al., 2023), which is publicly available, so it is possible the evaluated model encountered the original problems during training. We discuss this contamination risk in the Limitations section. We introduce no new human-subjects data, and the benchmark contains only algorithmic programming problems with no personal, sensitive, or identifying information.

Finally, our operational definition of intent reflects the judgment of a single annotator constructing the ambiguous and gold prompts and the test suites. Any measure of intent is inherently normative, and our benchmark encodes one reasonable interpretation of each problem rather than a ground truth. We therefore present IVR as a relative diagnostic, not an absolute measure of whether LLM-generated code is "correct".

References

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. [Program synthesis with large language models](#). *arXiv preprint arXiv:2108.07732*.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. [Evaluating large language models trained on code](#). *arXiv preprint arXiv:2107.03374*.
- Jonathan Gabor, Jayson Lynch, and Jonathan Rosenfeld. 2025. [EvilGenie: A reward hacking benchmark](#). *arXiv preprint arXiv:2511.21654*.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. [LiveCodeBench: Holistic and contamination free evaluation of large language models for code](#). *arXiv preprint arXiv:2403.07974*.
- Hyuhng Joon Kim, Youna Kim, Cheonbok Park, Junyeob Kim, Choonghyun Park, Kang Min Yoo, Sang-goo Lee, and Taeuk Kim. 2024. [Aligning language models to explicitly handle ambiguity](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1989–2007, Miami, Florida, USA. Association for Computational Linguistics.

Jialin Li, Yuan Wu, and Yi Chang. 2026. **ClarEval: A benchmark for evaluating clarification skills of code agents under ambiguous instructions**. *arXiv preprint arXiv:2603.00187*.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. **Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation**. In *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572.

Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binqun Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. **ClarifyGPT: Empowering LLM-based code generation with intention clarification**. *arXiv preprint arXiv:2310.10996*.

Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. 2025. **ImpossibleBench: Measuring LLMs’ propensity of exploiting test cases**. *arXiv preprint arXiv:2510.20270*.

A Spec Pair Construction Details

Construction Protocol: - Start from a HumanEval+ problem and its original prompt - Identify the implicit constraints the original problem specifies (ordering, edge-case handling, conditional behavior, etc.) - Write a gold (clarified) prompt built off the original prompt that states all constraints explicitly; in some cases add specifics beyond the original to remove residual ambiguity - Strip the constraints from the gold prompt to produce the ambiguous prompt; omitting information that does not steer toward a wrong interpretation - Encode each stripped constraint as a hidden test (C2–C4); C1 is a stated test that must be satisfiable from the ambiguous prompt alone - Problems are algorithm only as they cleanly decompose into executable tests; other task types involve fuzzier constraints and lack a comparably licensed source at scale

HumanEval/88 (conditional sort, IVR = 1.0)

Ambig.	Return a sorted copy of a list of non-negative integers, without modifying the original.
Gold	Sort ascending if the sum of the first and last elements is odd, descending if even; do not mutate the input.
C1	<i>stated</i> : sorts (odd sum) solution([2, 4, 3, 0, 1, 5]) == [0, 1, 2, 3, 4, 5]
C2	<i>hidden</i> : descending on even sum solution([2, 4, 3, 0, 1, 5, 6]) == [6, 5, 4, 3, 2, 1, 0]

Table 4: HE/88: the model sorts but ignores the parity-dependent direction. All 5 solutions violate C2.

HumanEval/161 (fallback, IVR = 1.0)

Ambig.	Swap the case of each letter, leaving non-letter characters unchanged.
Gold	Swap the case of each letter; if the string contains no letters, return it reversed instead.
C1	<i>stated</i> : case swap solution('#a@c') == '#A@c'
C2	<i>hidden</i> : reverse if no letters solution('1234') == '4321'

Table 5: HE/161: the model performs the case swap but misses the no-letters fallback. All 5 solutions violate C2.

HumanEval/25 (satisfied, IVR = 0.0)

Ambig.	Return the prime factors of a given integer.
Gold	Return the prime factors in ascending order, with repeated factors and product equal to the input.
C1	<i>stated</i> : basic factorization solution(13) == [13]
C2	<i>hidden</i> : ascending order solution(15) == [3, 5]

Table 6: HE/25: the ascending-order constraint is satisfied by all 5 solutions, as standard factorization emits factors in ascending order. Contrast with HE/34, where the analogous ordering constraint is violated.

Validation Detail

The benchmark was validated in two passes.

Leakage audit. Each stated test (C1) must be satisfiable from the ambiguous prompt alone, without knowledge of any hidden constraint. Of the 49 problems, 9 initially failed this check: type errors, output-format contamination (e.g., a list expected where a tuple was returned), and hidden-constraint leakage, where C1 inadvertently tested a hidden constraint so that no solution could pass C1 without already encoding part of the intent. All 9 were corrected.

Canonical soundness check. We ran the HumanEval+ canonical solution for each problem against the full test suite (C1 and all hidden constraints); a correct solution must pass all constraints, yielding IVR = 0. This surfaced 2 further problems whose tests were themselves faulty rather than leaked: HE/47 had an incorrect expected median value, and HE/18 contained a contradictory empty-substring constraint. Both were corrected (the HE/18 constraint was removed, as the empty-substring convention is ambiguous). After correction, all 49 canonical references yield IVR = 0,

confirming the hidden tests are satisfiable by correct code.

B Prompts and Sandbox Configuration

The generation prompt template is:

Write a Python function named ``solution`` that solves the following task.

{prompt}

Requirements:

- Name the function exactly ``solution``.
- Return only the function definition – no explanation, no markdown, no imports unless the function itself needs them.

where {prompt} is the ambiguous prompt.

Decoding Parameters: - Models: Claude Sonnet 4.6 and OpenAI GPT-4.1 - Temperature: 1.0 (API default) - Max Tokens: 1024 - Number Samples per Problem: 5

Post Processing: Markdown code fences are stripped from model output before execution to prevent syntax errors from non-code formatting.

Sandbox: Subprocess-based execution, one subprocess per test, 5-second timeout per test, adapted from HumanEval’s execution harness (process isolation, syscall restrictions). Each solution is run against C1 (stated test) and each hidden constraint independently.

C Claude Sonnet 4.6 Per-Problem Results

Table 7 reports the per-problem IVR for all 49 spec pairs, the number of solutions passing the stated test (C1) out of 5, and which hidden constraints were violated. HE/101 and HE/112 had no C1-passing solutions and are excluded from the IVR mean.

D OpenAI GPT-4.1 Per-Problem Results

Table 8 reports the per-problem IVR for GPT-4.1 across all 49 spec pairs. HE/21, HE/108, and HE/112 had no C1-passing solutions and are excluded from the IVR mean.

Problem	C1 pass	IVR	Violated
HE/7	5/5	0.00	–
HE/12	5/5	0.40	C3
HE/18	5/5	0.00	–
HE/20	5/5	0.00	–
HE/21	5/5	0.00	–
HE/25	5/5	0.00	–
HE/26	5/5	0.00	–
HE/30	5/5	0.00	–
HE/34	5/5	1.00	C2
HE/42	5/5	0.00	–
HE/47	5/5	0.00	–
HE/51	5/5	0.00	–
HE/57	5/5	1.00	C2
HE/58	5/5	1.00	C2,C3
HE/64	5/5	1.00	C2,C3
HE/65	5/5	1.00	C3
HE/66	5/5	1.00	C2
HE/68	5/5	0.00	–
HE/70	5/5	0.00	–
HE/71	5/5	1.00	C2,C3
HE/72	5/5	1.00	C2
HE/74	5/5	0.00	–
HE/86	5/5	1.00	C2,C3
HE/88	5/5	1.00	C2
HE/90	5/5	1.00	C3,C4
HE/91	5/5	1.00	C2,C3
HE/93	5/5	1.00	C2,C3
HE/95	5/5	1.00	C4
HE/98	5/5	0.00	–
HE/99	5/5	0.00	–
HE/101	0/5	–	–
HE/102	5/5	1.00	C3
HE/104	5/5	1.00	C2
HE/105	5/5	0.20	C2
HE/108	1/5	0.00	–
HE/111	5/5	1.00	C2,C3
HE/112	0/5	–	–
HE/120	5/5	0.00	–
HE/126	5/5	1.00	C3
HE/131	5/5	0.00	–
HE/135	5/5	0.00	–
HE/136	5/5	0.00	–
HE/140	5/5	1.00	C2
HE/145	5/5	1.00	C2
HE/149	5/5	1.00	C2
HE/151	5/5	1.00	C2
HE/158	5/5	1.00	C2
HE/161	5/5	1.00	C2
HE/163	5/5	1.00	C2,C3

Table 7: Per-problem IVR across all 49 spec pairs. “C1 pass” is the number of the 5 sampled solutions passing the stated test; “IVR” is the fraction of those that violate ≥ 1 hidden constraint; “Violated” lists the hidden constraints failed by any C1-passing solution.

Problem	C1 pass	IVR	Violated
HE/7	5/5	0.00	–
HE/12	5/5	1.00	C3
HE/18	5/5	0.80	C2
HE/20	5/5	0.00	–
HE/21	0/5	–	–
HE/25	5/5	0.00	–
HE/26	5/5	0.00	–
HE/30	5/5	0.00	–
HE/34	3/5	1.00	C2
HE/42	5/5	0.00	–
HE/47	5/5	0.00	–
HE/51	5/5	0.00	–
HE/57	5/5	1.00	C2
HE/58	5/5	1.00	C3
HE/64	5/5	1.00	C2,C3
HE/65	5/5	1.00	C3
HE/66	5/5	1.00	C2
HE/68	5/5	0.80	C3,C4
HE/70	5/5	0.00	–
HE/71	5/5	1.00	C2,C3
HE/72	5/5	1.00	C2
HE/74	5/5	0.40	C2
HE/86	5/5	1.00	C2,C3
HE/88	5/5	1.00	C2
HE/90	5/5	1.00	C3,C4
HE/91	5/5	1.00	C2,C3
HE/93	5/5	1.00	C2,C3
HE/95	5/5	1.00	C4
HE/98	5/5	0.00	–
HE/99	5/5	0.00	–
HE/101	4/5	1.00	C2,C3
HE/102	5/5	1.00	C3
HE/104	5/5	1.00	C2
HE/105	5/5	0.20	C2
HE/108	0/5	–	–
HE/111	5/5	1.00	C2
HE/112	0/5	–	–
HE/120	5/5	0.00	–
HE/126	5/5	1.00	C3
HE/131	5/5	0.00	–
HE/135	5/5	0.00	–
HE/136	5/5	0.00	–
HE/140	5/5	1.00	C2,C3
HE/145	5/5	1.00	C2
HE/149	5/5	1.00	C2
HE/151	5/5	1.00	C2
HE/158	5/5	1.00	C2
HE/161	5/5	1.00	C2
HE/163	5/5	1.00	C2,C3

Table 8: Per-problem IVR for GPT-4.1 across all 49 spec pairs. “C1 pass” is the number of the 5 sampled solutions passing the stated test; “IVR” is the fraction of those that violate ≥ 1 hidden constraint; “Violated” lists the hidden constraints failed by any C1-passing solution.